

Judge TCG — Cursor Context File v2 (Corrigido)

Generated: 2026-06-06

Repository: S:-judge

Status: Produção (Judge RAG) + Plataforma de Torneios (Fases 1–5 ~80–85%)

Testes: 107/109 vitest OK | pytest bloqueado por importação em public_api/auth.py

1. CORREÇÕES CRÍTICAS IDENTIFICADAS

1.1 Causa raiz do pytest bloqueado

Arquivo: services/api/app/public_api/auth.py **Problema:** require_api_key usa session: AsyncSession sem Depends **Impacto:** Impede importar app.main → quebra toda a suite pytest **Fix:** Usar DbSession (Annotated[AsyncSession, Depends(get_db_session)])

```
# services/api/app/public_api/auth.py

from typing import Any, Annotated
from fastapi import Header, Depends, HTTPException, status
from sqlalchemy.ext.asyncio import AsyncSession

from app.db import get_db_session
from app.public_api.validation import validate_api_key

DbSession = Annotated[AsyncSession, Depends(get_db_session)]

async def require_api_key(
    session: DbSession,
    x_api_key: str | None = Header(default=None, alias="X-API-Key"),
) -> dict[str, Any]:
    if not x_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="X-API-Key header is required"
        )

    key_data = await validate_api_key(session, x_api_key)
    if not key_data:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid API key"
        )
```

```
return key_data
```

1.2 games.py — mesmo padrão antigo

Verificar: services/api/app/tcg_adapters/games.py ou similar Se tiver **AsyncSession = Depends(...)**
→ corrigir para DbSession

1.3 Nomes reais dos routers (main.py)

Nomes documentados errados no PDF anterior:

Documentado	Nome Real em main.py
players_router	players_ecosystem_router
marketplace_router	marketplace_api_router
stores_router	stores_api_router (ou similar —verificar main.py)
social_router	social_api_router (ou similar)

Verificar em: services/api/app/main.py

1.4 Localizações corrigidas

Item no PDF anterior	Localização Real
judge-panel/ em services/api/app/	ERRADO —está em frontend/ runtime_console_v3/src/components/ judge-panel/
/judge = painel juiz Fase 6	ERRADO —/judge é o Judge RAG ; painel operacional está em /judge/dashboard
DecklistDetail parcial	Não existe —só DecklistCard
StorecreateForm ausente	Confirmado —não há ficheiro

2. SPRINT 0 — DESBLOQUEAR BASE (1–2 dias)

Tarefa 1: Corrigir **require_api_key** em **public_api/auth.py**

```
# services/api/app/public_api/auth.py

from typing import Any, Annotated
from fastapi import Header, Depends, HTTPException, status
from sqlalchemy.ext.asyncio import AsyncSession

from app.db import get_db_session
from app.public_api.validation import validate_api_key

DbSession = Annotated[AsyncSession, Depends(get_db_session)]

async def require_api_key(
```

```

    session: DbSession,
    x_api_key: str | None = Header(default=None, alias="X-API-Key"),
) -> dict[str, Any]:
    if not x_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="X-API-Key header is required"
        )

    key_data = await validate_api_key(session, x_api_key)
    if not key_data:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid API key"
        )

    return key_data

```

Tarefa 2: Verificar e corrigir games.py (se aplicável)

```

# services/api/app/tcg_adapters/games.py (ou similar)
# Verificar se há o mesmo padrão:

# Se encontrar:
# async def get_games(session: AsyncSession = Depends(get_db)):

# Corrigir para:
from app.db import DbSession

async def get_games(session: DbSession):
    ...

```

Tarefa 3: Fix OnboardingFlow.test.tsx

```

// frontend/runtime_console_v3/src/components/onboarding/__tests__/OnboardingFlow.test.tsx

import { render, screen, fireEvent } from '@testing-library/react';
import { OnboardingFlow } from '../OnboardingFlow';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

jest.mock('@components/tournament/GameSelector', () => ({
  GameSelector: ({ onSelect }: { onSelect: (game: string) => void }) => (
    <button data-testid="game-selector" onClick={() => onSelect('MTG')}>
      Select MTG
    </button>
  )
}));

jest.mock('@hooks/useTournamentGames', () => ({
  useTournamentGames: () => ({
    games: [

```

```

    { code: 'MTG', name: 'Magic: The Gathering' },
    { code: 'POKEMON', name: 'Pokémon TCG' }
  ],
  isLoading: false
})
}));

const renderWithProviders = (ui: React.ReactElement) => {
  const queryClient = new QueryClient({
    defaultOptions: { queries: { retry: false } }
  });
  return render(
    <QueryClientProvider client={queryClient}>
      {ui}
    </QueryClientProvider>
  );
};

describe('OnboardingFlow', () => {
  it('completes all 5 steps', async () => {
    renderWithProviders(<OnboardingFlow />);

    expect(screen.getByText(/Bem-vindo/)).toBeInTheDocument();
    fireEvent.click(screen.getByText(/Próximo/));

    expect(screen.getByText(/Crie seu Perfil/)).toBeInTheDocument();
  });
});
});

```

Tarefa 4: Remover .pem do working tree

```

cd S:\tcg-judge

# Verificar quais .pem existem:
git ls-files | grep "\.pem$"

# Remover do git (não do disco se necessário para local):
git rm --cached judge-tcg-key.pem keysson.pem sgka.pem

# Adicionar ao .gitignore se ainda não estiver:
echo "*.pem" >> .gitignore
echo "*.key" >> .gitignore
echo "service-account*.json" >> .gitignore

git add .gitignore
git commit -m "security: remove .pem files from tracking, update .gitignore"

```

3. SPRINT 1 — COMPLETAR PLATAFORMA DE TORNEIOS (~1 semana)

3.1 Rotas Frontend Ausentes

Rota	Componente	API/BFF	Prioridade
/player/me	PlayerProfileEditForm	PUT /runtime/judge/ players/me	Alta
/stores	StoreListPage	GET /runtime/judge/ stores	Alta
/stores/create	StoreCreateForm	POST /runtime/judge/ stores	Alta
/marketplace/ [id]	DecklistDetail	GET /runtime/judge/ marketplace/decklists/ {id}	Média
/social/ friends	FriendsListPage	GET /runtime/judge/ social/friends	Média
/social/ communities	CommunitiesListPage	GET /runtime/judge/ social/communities	Média

3.2 Código-base para /player/me

```
// frontend/runtime_console_v3/src/app/player/me/page.tsx
'use client';

import { useState } from 'react';
import { usePlayerProfile } from '@/hooks/usePlayerProfile';
import { useUpdateProfile } from '@/hooks/useUpdateProfile';
import { AvatarUpload } from '@/components/player/AvatarUpload';
import { NotificationPreferences } from '@/components/notifications/NotificationPreferences';

export default function MyProfilePage() {
  const { profile, isLoading } = usePlayerProfile('me');
  const { updateProfile, isUpdating } = useUpdateProfile();
  const [formData, setFormData] = useState({
    displayName: '',
    bio: '',
    favoriteGame: '',
    city: '',
    country: '',
  });

  if (isLoading) return <div>Carregando...</div>;

  return (
    <div className="max-w-2xl mx-auto py-8">
      <h1 className="text-2xl font-bold mb-6">Meu Perfil</h1>

      <div className="space-y-6">
        <section className="bg-white rounded-lg shadow p-6">
          <h2 className="text-lg font-semibold mb-4">Foto de Perfil</h2>
```

```

    <AvatarUpload currentUrl={profile?.avatarUrl} />
  </section>

  <section className="bg-white rounded-lg shadow p-6">
    <h2 className="text-lg font-semibold mb-4">Informações</h2>
    <form
      onSubmit={(e) => {
        e.preventDefault();
        updateProfile(formData);
      }}
      className="space-y-4"
    >
      <div>
        <label className="block text-sm font-medium mb-1">Nome de exibição</label>
        <input
          type="text"
          value={formData.displayName}
          onChange={(e) => setFormData({ ...formData, displayName: e.target.value })}
          className="w-full border rounded-lg px-3 py-2"
          placeholder={profile?.displayName}
        />
      </div>

      <div>
        <label className="block text-sm font-medium mb-1">Bio</label>
        <textarea
          value={formData.bio}
          onChange={(e) => setFormData({ ...formData, bio: e.target.value })}
          className="w-full border rounded-lg px-3 py-2"
          rows={3}
          placeholder={profile?.bio || 'Conte um pouco sobre você...'}
        />
      </div>

      <div className="grid grid-cols-2 gap-4">
        <div>
          <label className="block text-sm font-medium mb-1">Cidade</label>
          <input
            type="text"
            value={formData.city}
            onChange={(e) => setFormData({ ...formData, city: e.target.value })}
            className="w-full border rounded-lg px-3 py-2"
          />
        </div>
        <div>
          <label className="block text-sm font-medium mb-1">País</label>
          <input
            type="text"
            value={formData.country}
            onChange={(e) => setFormData({ ...formData, country: e.target.value })}
            className="w-full border rounded-lg px-3 py-2"
          />
        </div>
      </div>
    </form>
  </section>

```

```

        />
      </div>
    </div>

    <button
      type="submit"
      disabled={isUpdating}
      className="w-full bg-blue-600 text-white py-2 rounded-lg hover:bg-blue-700
        ↪ disabled:bg-gray-300"
    >
      {isUpdating ? 'Salvando...' : 'Salvar Alterações'}
    </button>
  </form>
</section>

<section className="bg-white rounded-lg shadow p-6">
  <h2 className="text-lg font-semibold mb-4">Notificações</h2>
  <NotificationPreferences />
</section>
</div>
</div>
);
}

```

3.3 Código-base para /stores/create

```

// frontend/runtime_console_v3/src/app/stores/create/page.tsx
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import { useCreateStore } from '@/hooks/useCreateStore';

interface StoreFormData {
  name: string;
  slug: string;
  description: string;
  address: string;
  city: string;
  state: string;
  country: string;
  phone: string;
  email: string;
  website: string;
  discord: string;
}

export default function CreateStorePage() {
  const router = useRouter();
  const { createStore, isLoading, error } = useCreateStore();
  const [formData, setFormData] = useState<StoreFormData>({

```

```

    name: '',
    slug: '',
    description: '',
    address: '',
    city: '',
    state: '',
    country: 'BR',
    phone: '',
    email: '',
    website: '',
    discord: '',
  });

const generateSlug = (name: string) => {
  return name
    .toLowerCase()
    .replace(/[a-z0-9]+/g, '-')
    .replace(/^-|-$/g, '');
};

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  try {
    const store = await createStore(formData);
    router.push(`/stores/${store.slug}`);
  } catch (err) {
    console.error('Failed to create store:', err);
  }
};

return (
  <div className="max-w-2xl mx-auto py-8">
    <h1 className="text-2xl font-bold mb-6">Criar Nova Loja</h1>

    <form onSubmit={handleSubmit} className="space-y-6">
      <div className="bg-white rounded-lg shadow p-6 space-y-4">
        <h2 className="text-lg font-semibold">Informações Básicas</h2>

        <div>
          <label className="block text-sm font-medium mb-1">Nome da Loja *</label>
          <input
            type="text"
            required
            value={formData.name}
            onChange={(e) => {
              const name = e.target.value;
              setFormData({
                ...formData,
                name,
                slug: generateSlug(name)
              });
            }}
          />
        </div>
      </div>
    </form>
  </div>
);

```

```

    }}
    className="w-full border rounded-lg px-3 py-2"
    placeholder="Ex: Loja TCG Central"
  />
</div>

<div>
  <label className="block text-sm font-medium mb-1">Slug (URL) *</label>
  <div className="flex items-center">
    <span className="text-gray-500 mr-2">tcg-judge.com/stores/</span>
    <input
      type="text"
      required
      value={formData.slug}
      onChange={(e) => setFormData({ ...formData, slug: e.target.value })}
      className="flex-1 border rounded-lg px-3 py-2"
      pattern="[a-z0-9-]+"
    />
  </div>
</div>

<div>
  <label className="block text-sm font-medium mb-1">Descrição</label>
  <textarea
    value={formData.description}
    onChange={(e) => setFormData({ ...formData, description: e.target.value })}
    className="w-full border rounded-lg px-3 py-2"
    rows={3}
    placeholder="Descreva sua loja..."
  />
</div>
</div>

<div className="bg-white rounded-lg shadow p-6 space-y-4">
  <h2 className="text-lg font-semibold">Localização</h2>

  <div>
    <label className="block text-sm font-medium mb-1">Endereço</label>
    <input
      type="text"
      value={formData.address}
      onChange={(e) => setFormData({ ...formData, address: e.target.value })}
      className="w-full border rounded-lg px-3 py-2"
    />
  </div>

  <div className="grid grid-cols-3 gap-4">
    <div>
      <label className="block text-sm font-medium mb-1">Cidade</label>
      <input
        type="text"

```

```

        value={formData.city}
        onChange={(e) => setFormData({ ...formData, city: e.target.value })}
        className="w-full border rounded-lg px-3 py-2"
      />
    </div>
    <div>
      <label className="block text-sm font-medium mb-1">Estado</label>
      <input
        type="text"
        value={formData.state}
        onChange={(e) => setFormData({ ...formData, state: e.target.value })}
        className="w-full border rounded-lg px-3 py-2"
      />
    </div>
    <div>
      <label className="block text-sm font-medium mb-1">País</label>
      <input
        type="text"
        value={formData.country}
        onChange={(e) => setFormData({ ...formData, country: e.target.value })}
        className="w-full border rounded-lg px-3 py-2"
      />
    </div>
  </div>
</div>

<div className="bg-white rounded-lg shadow p-6 space-y-4">
  <h2 className="text-lg font-semibold">Contato</h2>

  <div className="grid grid-cols-2 gap-4">
    <div>
      <label className="block text-sm font-medium mb-1">Telefone</label>
      <input
        type="tel"
        value={formData.phone}
        onChange={(e) => setFormData({ ...formData, phone: e.target.value })}
        className="w-full border rounded-lg px-3 py-2"
      />
    </div>
    <div>
      <label className="block text-sm font-medium mb-1">Email *</label>
      <input
        type="email"
        required
        value={formData.email}
        onChange={(e) => setFormData({ ...formData, email: e.target.value })}
        className="w-full border rounded-lg px-3 py-2"
      />
    </div>
  </div>
</div>

```

```

<div className="grid grid-cols-2 gap-4">
  <div>
    <label className="block text-sm font-medium mb-1">Website</label>
    <input
      type="url"
      value={formData.website}
      onChange={(e) => setFormData({ ...formData, website: e.target.value })}
      className="w-full border rounded-lg px-3 py-2"
      placeholder="https://..."
    />
  </div>
  <div>
    <label className="block text-sm font-medium mb-1">Discord</label>
    <input
      type="text"
      value={formData.discord}
      onChange={(e) => setFormData({ ...formData, discord: e.target.value })}
      className="w-full border rounded-lg px-3 py-2"
      placeholder="discord.gg/..."
    />
  </div>
</div>

{error && (
  <div className="bg-red-50 border border-red-200 text-red-700 p-4 rounded-lg">
    {error.message}
  </div>
)}

<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-blue-600 text-white py-3 rounded-lg font-semibold hover:bg-blue-700
  ↵ disabled:bg-gray-300"
>
  {isLoading ? 'Criando...' : 'Criar Loja'}
</button>
</form>
</div>
);
}

```

4. SPRINT 2 — QUALIDADE E INTEGRAÇÃO (~1 semana)

4.1 Consolidar pytest

```

cd S:\tcg-judge
mkdir -p tests/platform

```

```
# Mover/criar testes para:
tests/platform/
├─ test_tournament_engine.py      # Swiss, tiebreakers, bracket
├─ test_tcg_adapters.py          # Parsers, validation
├─ test_players.py               # Profile, rankings
├─ test_social.py                # Friends, messages
├─ test_marketplace.py           # Decklists, sales
├─ test_stores.py                # CRUD lojas
└─ test_public_api.py            # API key auth, rate limit

pytest tests/platform/ -v --tb=short
```

4.2 E2E mínimo

```
Cenário: Criar torneio → Check-in → Rodada → Report → Standings

1. Organizador cria torneio (POST /tournaments)
2. Jogadores se inscrevem (POST /tournaments/{id}/register)
3. Organizador abre check-in (POST /tournaments/{id}/start-check-in)
4. Jogadores fazem check-in
5. Organizador inicia torneio (POST /tournaments/{id}/start)
6. Sistema gera rodada 1 (POST /tournaments/{id}/rounds)
7. Organizador inicia timer (POST /rounds/{n}/start)
8. Jogadores reportam resultados (POST /pairings/{id}/report)
9. Jogadores confirmam (POST /pairings/{id}/confirm)
10. Organizador finaliza rodada (POST /rounds/{n}/end)
11. Standings recalculados (GET /tournaments/{id}/standings)
```

4.3 Checklist de deploy

- API no Render: health check OK
- Frontend no Vercel: build OK
- Supabase: migrations aplicadas, RLS ativo
- Redis: conectado, timer funcionando
- Stripe: webhooks configurados, secrets no env
- VAPID keys: geradas, no env
- Domínio: apontando para Vercel
- SSL: HTTPS forçado

5. DECISÃO ESTRATÉGICA — FASE 6

Opção	Estado Atual	Prós	Contras	Recomendação
Painel Juiz Digital	Schema infractions existe, componentes judge-panel/ existem, API judge_assistant existe	Menor esforço, aproveita infra	Menor reach que mobile	PRIORIZAR

Table 4 – continued

Opção	Estado Atual	Prós	Contras	Recomendação
Mobile Expo	apps/mobile/ estrutura extensa	Maior reach, alinhado README	Não é app único de produção, maior esforço	Depois
POS	Zero implementação	Integração física	Maior esforço, menor urgência	Depois

Recomendação: Painel Juiz Digital (aproveita schema e componentes existentes).

6. REGRAS DE OURO PARA NOVOS PRs

6.1 Classificação antes de editar

```

É Judge RAG?      → runtime_judge.py, retrieval/, judge/, /judge, ingestão
É Torneio?        → tournament/, tcg_adapters/, /tournament/*, marketplace
É Enterprise?     → generated/, infra/aws/, admin_console_v2/ → EVITAR

```

6.2 Não quebrar Judge RAG

- Alterações na plataforma de torneios NÃO devem quebrar /runtime/judge/query
- Não alterar main.py sem verificar se routers do RAG ainda funcionam
- Proxy Vercel /api/proxy/* deve continuar funcionando

6.3 CI = prioridade máxima

- Nunca mergear com testes quebrados
- pytest deve passar antes de PR
- vitest deve passar antes de PR

7. CHECKLIST “PRONTO PARA CONTINUAR”

- require_api_key corrigido → pytest -co sem erro
- vitest 109/109 verde
- .pem removidos do working tree
- 6 rotas frontend ausentes criadas
- Decisão Fase 6 documentada (juiz vs. mobile vs. POS)
- E2E torneio completo validado
- Deploy Render/Vercel/Supabase verificado

8. COMANDOS RÁPIDOS

```

# Verificar estado dos testes
pytest tests/ --co -q           # Coleta sem executar
pytest tests/platform/ -v       # Suite plataforma

```

```
npm run test -- --run                # Vitest (frontend)

# Verificar .pem
git ls-files | grep "\.pem$"          # Listar .pem trackeados

# Verificar routers
python -c "from app.main import app; print([r.path for r in app.routes])"

# Verificar migrations
supabase migration list                # Listar migrations
supabase db push                       # Aplicar pendentes
```

Este arquivo reflete o ESTADO REAL corrigido do monorepo em 2026-06-06. Atualizar após Sprint 0 (correções críticas).